

again from the beginning (A). Just like on a clock, after 12 comes 1, and after 'Z', we return to 'A'. Thus, after applying the Caesar cipher with a shift of three, the original message 'I READ OSFY' becomes 'L UHGD RVIB'.

To decrypt the ciphertext, we reverse the process by shifting each letter three positions backward in the alphabet. Let us see how the ciphertext 'L UHGD RVIB' is decrypted. The letter 'L' shifts back to 'I' ($L \rightarrow K \rightarrow J \rightarrow I$), 'U' shifts back to 'R' ($U \rightarrow T \rightarrow S \rightarrow R$), 'H' shifts back to 'E' ($H \rightarrow G \rightarrow F \rightarrow E$), and so on. Finally, the decrypted message obtained is 'I READ OSFY', which is the same as the original plaintext.

Let us now implement the Caesar cipher with a shift of three using SageMath. The program 'caesar20.sagews', shown below, implements Caesar cipher (line numbers are included for clarity).

```
1. def caesar_cipher(text, shift):
2.     result = ""
3.     for char in text:
4.         if char.isalpha():
5.             new_pos = (ord(char)-ord('A')+shift)%26
6.             new_char = chr(new_pos + ord('A'))
7.             result += new_char
8.         else:
9.             result += char
10.    return result
```

Now, let us go through the code line by line to understand how it works. Line 1 defines a function named *caesar_cipher()* that takes two arguments: *text* (the string to be encrypted or decrypted) and *shift* (the number of positions each letter will be shifted in the alphabet). Line 2 initialises an empty string called *result*, where the encrypted text will be stored as the function processes each character from the input text. Line 3 starts a loop that iterates over each character in the input text. Line 4 checks if the current character is an alphabetic letter (A-Z or a-z). If the character is a letter, it will be shifted; if not (for example, a space or punctuation), it will be left unchanged. In Line 5, *ord(char)* gets the Unicode (also ASCII) value of the current character. For example, *ord('A')* returns 65. The value given by *ord('A')* is subtracted to convert the character's ASCII value into a range that starts from 0 for 'A' (for example, 'A' is treated as 0, 'B' as 1, etc). Then, the value of the *shift* is added to shift the position of the character in the alphabet. The modulus operation (%) by 26 ensures that the result wraps around if it goes beyond 'Z'. Line 6 converts the shifted position back to the corresponding ASCII value and then converts this ASCII value back to a character and stores it in *new_char*. Line 7 adds the newly encrypted character in *new_char* to the string *result*. The *else* in Line 8 is executed only if the current character is not alphabetic. Line

9 adds this character unchanged to the string *result*. Once all characters have been processed, in Line 10, the function returns the string *result*, which contains the fully encrypted text or decrypted text.

Now, we need to test this Caesar cipher function using plaintext. The following lines of code are added to the end of the function *caesar_cipher()* to perform this testing (line numbers are included for clarity).

```
11. plaintext = "I READ OSFY"
12. shift = 3
13. encrypted_text = caesar_cipher(plaintext, shift)
14. print("Encrypted text:", encrypted_text)
15. decrypted_text = caesar_cipher(encrypted_text, -shift)
16. print("Decrypted text:", decrypted_text)
```

Let us try to understand how the testing works. Line 11 assigns the string 'I READ OSFY' to the variable *plaintext*. This is the text that we want to encrypt using the Caesar cipher. Line 12 sets the variable *shift* to 3, meaning each letter in the plaintext will be shifted by three positions in the alphabet during encryption. For example, 'A' becomes 'D', 'B' becomes 'E', and so on. Line 13 calls the function *caesar_cipher()* with two arguments: *plaintext* and *shift* (3). The function encrypts the plaintext by shifting each alphabetic character by three positions. The result of this encryption is stored in the variable *encrypted_text*. Line 14 prints the encrypted version of the plaintext. Line 15 decrypts the *encrypted_text* by calling the function *caesar_cipher()* again, but this time with the shift value as -3. The negative shift reverses the encryption, restoring the text to its original form. The decrypted text is stored in the variable *decrypted_text*. Line 16 prints the decrypted version of the encrypted text, which should match the original plaintext.

Now, let us execute the program 'caesar20.sagews'. As before, we will use the online tool called CoCalc to run the program. Upon execution, the following output will be displayed on the screen.

```
Encrypted text: L UHGD RVIB
Decrypted text: I READ OSFY
```

Notice that the encrypted text is neither meaningful nor similar to the plaintext. Now, let us modify Line 12 of the code to *shift = 4* and observe the output. Upon running the modified program, the encrypted text displayed is 'M VIEH SWJC', which is different from the encrypted text when the shift was 3. However, as before, the decrypted text remains 'I READ OSFY', matching the original plaintext. Next, we will try a shift of 26 by updating Line 12 to *shift = 26*. When the program is executed, the encrypted text displayed is 'I READ OSFY', identical to the plaintext. This occurs because a shift